Inline Function

When an ***inline function*** is called, the compiler replaces all the calling statements with the function definition at run-time. Every time an inline function is called, the compiler generates a copy of the function's code, in place, to avoid the function call.

Syntax:

```
inline return type function name(parameters)
{
    // function code
}
```

Example :

```
#include<iostream>
using namespace std;
inline int MAX(int a,int b)
{
    return (a > b)?a:b;
}
int main()
{
    cout<<MAX(10,20)<<endl;
    cout<<MAX(0,200)<<endl;
}
```

output :

```
20
200
```

Remember, inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining. Compiler may not perform inlining in such circumstances like:

Vector India

- If a function contains a loop. (for, while, do-while)
- If a function contains static variables.
- If a function is recursive.
- If a function return type is other than void, and the return statement doesn't exist in function body.
- If a function contains switch or goto statement.

Advantages of using inline functions

•Inline functions speed up your program by avoiding the overhead associated with function calls.

•Inline functions save the overhead of pushing and popping variables on the stack during function calls.

•Inline functions save the overhead of the return call from a function.

Vector India